

|                                                                                                               |                                                                           |                                     |                                |
|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------|--------------------------------|
| <br>UNIVERSIDAD<br>DON BOSCO | UNIVERSIDAD DON BOSCO<br>FACULTAD DE INGENIERÍA<br>ESCUELA DE COMPUTACIÓN |                                     | CICLO<br>02-2026               |
|                                                                                                               | MATERIA                                                                   | DESARROLLO DE SOFTWARE PARA MÓVILES | GUIA DE<br>LABORATORIO<br>N° 5 |
|                                                                                                               | PRACTICA                                                                  | RECURSOS                            |                                |

## I. OBJETIVOS

- 1 Conocer e incorporar diferentes tipos de recursos en Android.
- 2 Desarrollar aplicación android con acceso a varios recursos.

## II. INTRODUCCIÓN TEÓRICA

### Recursos

Los recursos son los archivos adicionales y el contenido estático que usa el código de la App, como mapas de bits, definiciones de diseño, strings de interfaz de usuario, instrucciones de animación, entre otros.

Siempre se debe externalizar los recursos para aplicaciones, como imágenes y strings en el código, para mantenerlos de forma independiente.

También se debe proporcionar recursos alternativos para configuraciones de dispositivos específicos, agrupándolos en directorios de recursos con un nombre especial.

En tiempo de ejecución, Android utiliza el recurso adecuado según la configuración actual. Por ejemplo, se puede proporcionar un diseño de interfaz de usuario (IU) diferente según el tamaño de la pantalla o strings diferentes según la configuración de idioma.

Una vez que externalizas los recursos para la aplicación, se puede acceder a ellos mediante los ID de recursos que se generan en la clase **R** del proyecto.

La gestión de recursos en Android con Kotlin implica manejar archivos como cadenas de texto, imágenes, colores y más para mantener la app organizada y facilitar su mantenimiento. Aquí hay un resumen de los conceptos clave y cómo gestionarlos:

### 1. Strings (Cadenas de texto)

Las cadenas de texto se almacenan en el archivo `res/values/strings.xml`.

```
<resources>
  <string name="app_name">Mi Aplicación</string>
  <string name="welcome_message">Bienvenido a mi app</string>
</resources>
```

Para acceder a estas cadenas en Kotlin:

```
val appName = getString(R.string.app_name)
val welcomeMessage = getString(R.string.welcome_message)
```

## 2. Colors (Colores)

Los colores se definen en res/values/colors.xml.

```
<resources>
    <color name="primaryColor">#6200EE</color>
    <color name="primaryDarkColor">#3700B3</color>
    <color name="accentColor">#03DAC5</color>
</resources>
```

Para acceder a los colores en Kotlin:

```
val primaryColor = ContextCompat.getColor(context, R.color.primaryColor)
```

## 3. Dimensiones

Las dimensiones como márgenes y tamaños de texto se guardan en res/values/dimens.xml.

```
<resources>
    <dimen name="activity_horizontal_margin">16dp</dimen>
    <dimen name="text_size">14sp</dimen>
</resources>
```

Para acceder a las dimensiones en Kotlin:

```
val margin = resources.getDimension(R.dimen.activity_horizontal_margin)
val textSize = resources.getDimension(R.dimen.text_size)
```

## 4. Imágenes y Drawables

Las imágenes se almacenan en las carpetas res/drawable.

Para acceder a una imagen en Kotlin:

```
val imageView: ImageView = findViewById(R.id.imageView)
imageView.setImageResource(R.drawable.image_name)
```

## 5. Estilos y Temas

Los estilos y temas se definen en `res/values/styles.xml`.

```
<resources>
  <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
    <item name="colorPrimary">@color/primaryColor</item>
    <item name="colorPrimaryDark">@color/primaryDarkColor</item>
    <item name="colorAccent">@color/accentColor</item>
  </style>
</resources>
```

## 6. Uso de Recursos en Código Kotlin

Para acceder a cualquier recurso en Kotlin, utilizas la clase `R` que genera automáticamente referencias a todos los recursos de la aplicación. Gestionar recursos correctamente en una aplicación Android es esencial para mantener el código limpio y fácil de mantener.

### Tipos de recursos.

Android posee 2 grandes tipos de recursos:

- **Recursos del proyecto.**  
Son recursos que son exclusivos del proyecto donde residen (recursos alternativos)
- **Recursos del sistema.**  
Son parte del sistema operativo y mantienen compatibilidad con las diferentes versiones del mismo.

### Recursos del proyecto.

Debes colocar cada tipo de recurso en un subdirectorío específico del directorio `res/` de tu proyecto. Por ejemplo, esta es la jerarquía de archivos de un proyecto simple:

```
MyProject/
  src/
    MainActivity.java
  res/
    drawable/
      graphic.png
    layout/
      main.xml
      info.xml
    mipmap/
      icon.png
    values/
      strings.xml
```

El directorio `res/` contiene todos los recursos (en subdirectorios), un recurso de imagen, dos recursos de diseño, directorios `mipmap/` para los íconos del selector y un archivo de recursos de strings.

Los nombres del directorio de recursos son importantes y se describen la siguiente tabla.

| Carpeta<br>(identificador<br>con clase R) | Descripción breve                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>res/drawable</b><br>R.drawable         | Archivos de mapas de bits (.png, .png, .jpg, .gif) o archivos XML que se han compilado en los siguientes subtipos de recursos de elemento de diseño: Archivos de mapas de bits, Nine patches (mapas de bits reajustables), <b>Listas</b> de estados, Formas, Elementos de diseño de animaciones, Otros elementos de diseño |
| <b>res/layout</b><br>R.layout             | Archivos XML que contienen layouts.                                                                                                                                                                                                                                                                                        |
| <b>res/menu</b><br>R.menu                 | Archivos XML con descriptores de menús que podemos usar en nuestra aplicación.                                                                                                                                                                                                                                             |
| <b>res/anim</b><br>R.anim                 | Archivos XML que permite definir una animación con interpolación de movimientos.                                                                                                                                                                                                                                           |
| <b>res/animator</b><br>R.animator         | Archivos XML que permiten modificar las propiedades de objetos a lo largo del tiempo.                                                                                                                                                                                                                                      |
| <b>res/xml</b><br>R.xml                   | Otros archivos XML como archivos de preferencias.                                                                                                                                                                                                                                                                          |
| <b>res/raw</b><br>R.raw                   | Archivos de Audio, video, texto, etc.                                                                                                                                                                                                                                                                                      |

| Archivos<br>(identificador<br>con<br>clase R) | Descripción breve                                                                        |
|-----------------------------------------------|------------------------------------------------------------------------------------------|
| res/values                                    | Archivos XML que definen un determinado valor para referencias un color, estilo, cadena. |

## Tipos de recursos en la carpeta values:

| Archivos<br>(identificador)      | Descripción breve                                                                                                                                                                                                                                                                         |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>strings.xml</b><br>R.string   | Identifica cadenas de caracteres<br><b>&lt;string name="hola"&gt;Hola mundo&lt;/string&gt;</b>                                                                                                                                                                                            |
| <b>colors.xml</b><br>R.color     | Colores definidos en formato ARGV<br>Los valores se indican en hexadecimal en los formatos #RGB, #ARGB, #RRGGBB ó #AARRGGBB<br><b>&lt;color name="verde_opaco"&gt;#0f0&lt;/color&gt;</b><br><b>&lt;color name="Rojo_translucido"&gt;#80ff0000&lt;/color&gt;</b>                           |
| <b>dimensions.xml</b><br>R.dimen | Un número seguido de una unidad de medida<br><b>px – pixeles, mm – milímetros, in – pulgadas, pt – puntos</b><br><b>dp – pixeles independientes de la densidad.</b><br><b>&lt;dimen name="alto"&gt;2.2mm&lt;/dimen&gt;</b><br><b>&lt;dimen name="tamaño_fuente"&gt;16dp&lt;/dimen&gt;</b> |

| Archivos<br>(identificador)   | Descripción breve                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>styles.xml</b><br>R.styles | Serie de atributos que pueden ser aplicados a una vista o actividad. Si se aplican a una actividad se conocen como temas.<br><b>&lt;style name="TextoGrande" parent="@style/Text"&gt;</b><br><b>&lt;item name="android:textSize"&gt;20pt&lt;/item&gt;</b><br><b>&lt;item name="android:textColor"&gt;#000080&lt;/item&gt;</b><br><b>&lt;/style&gt;</b> |
| R.int                         | Define un valor entero<br><b>&lt;integer name= "edad"&gt;20&lt;/integer&gt;</b>                                                                                                                                                                                                                                                                        |
| R.bool                        | Define un valor booleano<br><b>&lt;bool name= "continuar"&gt;true&lt;/bool&gt;</b>                                                                                                                                                                                                                                                                     |
| R.id                          | Define un identificador único de recurso.<br><b>&lt;item type="id" name="boton_ok" /&gt;</b><br><b>&lt;item type="id" name="txtNombre" /&gt;</b>                                                                                                                                                                                                       |

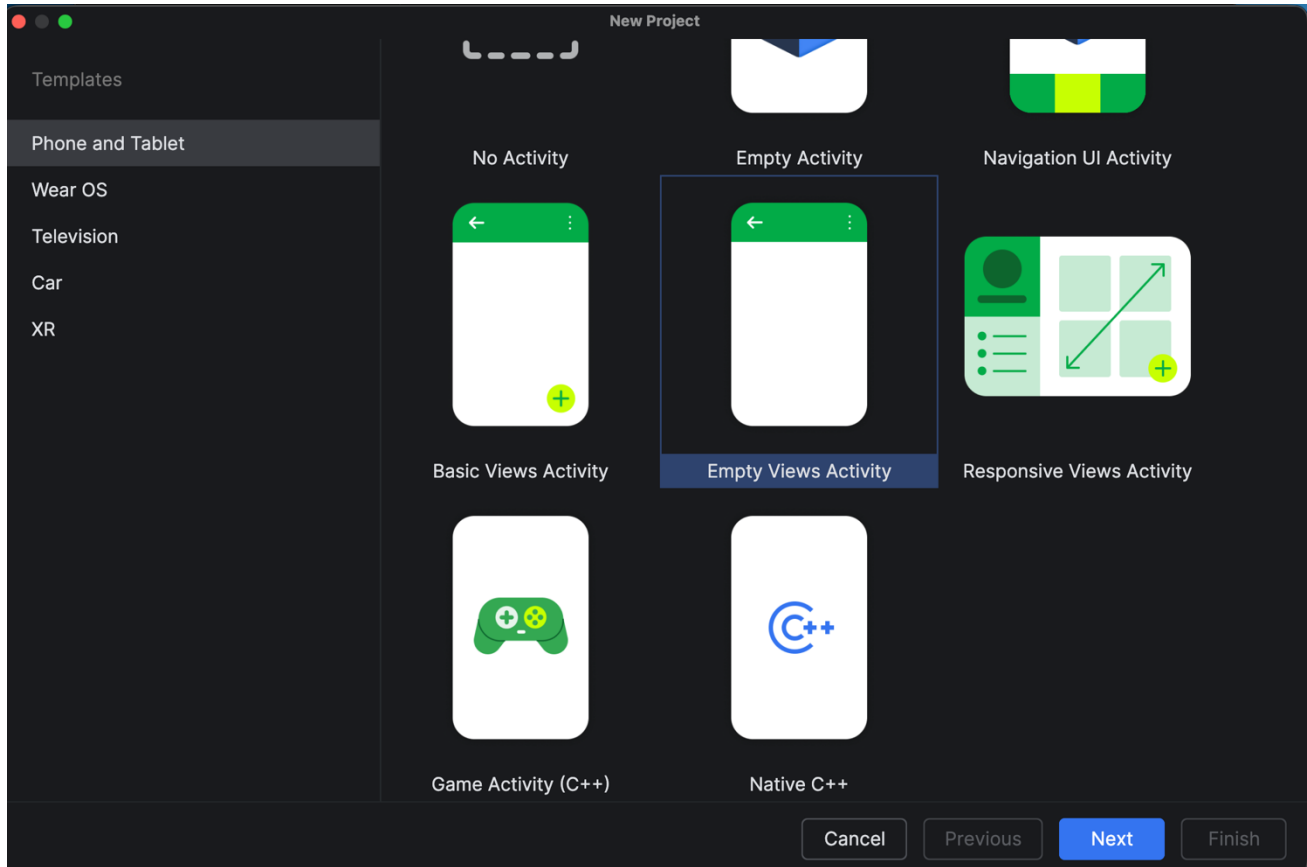
| Archivos<br>(identificador) | Descripción breve                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R.array                     | <p>Una serie ordenada de elementos. Pueden ser strings, enteros o de recursos (TypedArray)</p> <pre> &lt;string-array name= "dias_semana"&gt;   &lt;item&gt;lunes&lt;/item&gt;   &lt;item&gt;martes&lt;/item&gt; &lt;/string-array&gt; &lt;integer-array name= "meses"&gt;   &lt;item&gt;1&lt;/item&gt; &lt;item&gt;2&lt;/item&gt;&lt;item&gt;3&lt;/item&gt; &lt;/integer-array&gt; &lt;array name= "fotos"&gt;   &lt;item&gt;@drawable/foto1&lt;/item&gt;   &lt;item&gt;@drawable/foto2&lt;/item&gt; &lt;/array&gt; </pre> |

### III. PROCEDIMIENTO

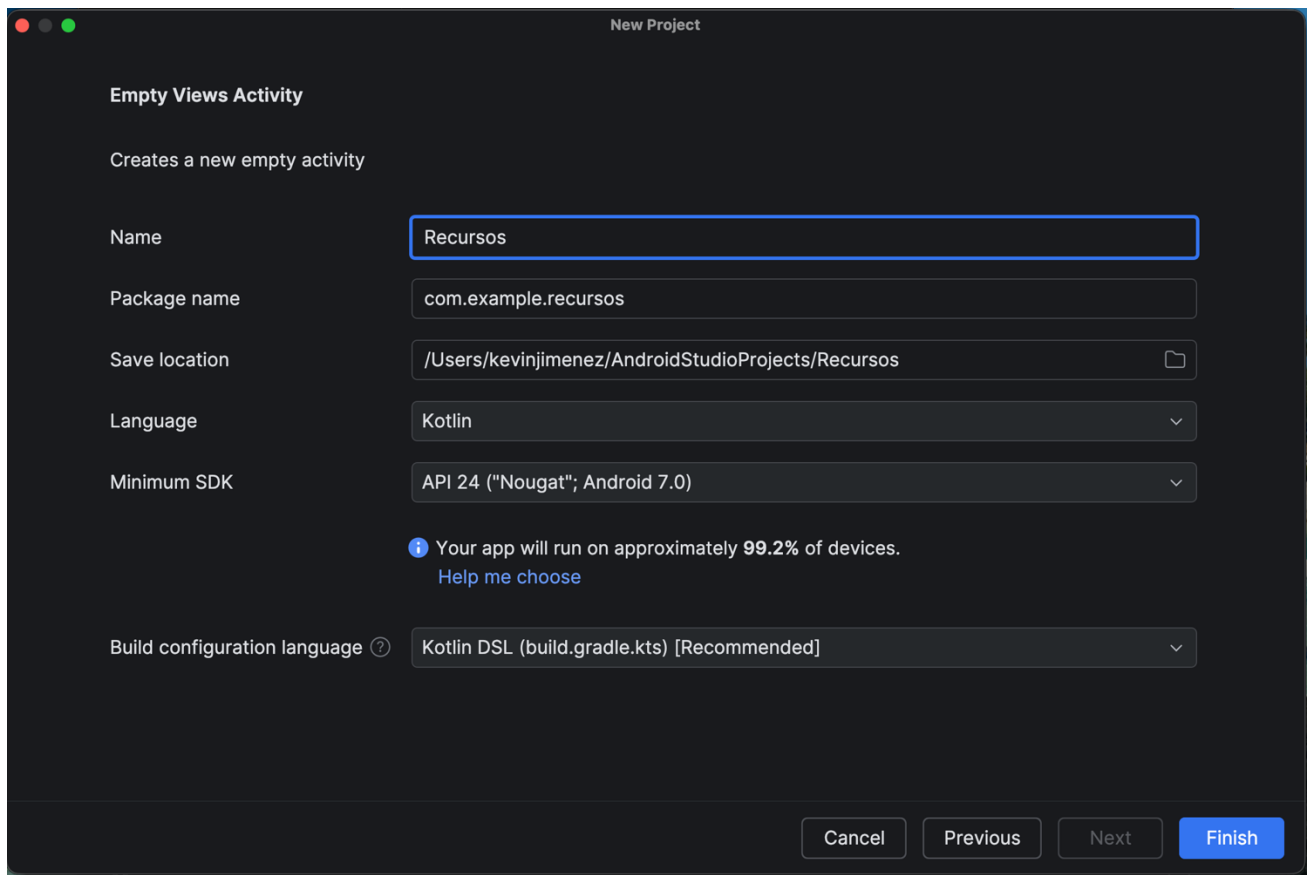
#### Gestión de Recursos Android

Se desarrollará una aplicación android donde se gestionará recursos de la carpeta **Drawable**, recursos **imagenes**, se gestionarán recursos **iconos**, recursos menú y se accederá al archivo **R.txt**.

- 1 Crear un nuevo proyecto Android, en New Project, seleccionar **Empty Views Activity**, y presiona **Next**.



- 2 En la siguiente ventana de New Project, configurar el proyecto según los siguientes detalles:



- 3 Para este proyecto agregaremos las librerías de *Recyclerview* y *Cardview* para construir una interfaz de catálogo en la aplicación.

build.gradle.kts (Module App)

```
android {  
    ...  
    buildFeatures {  
        viewBinding = true  
    }  
}  
  
dependencies {  
    ...  
    implementation("androidx.recyclerview:recyclerview:1.4.0")  
    implementation("androidx.cardview:cardview:1.0.0")  
    ...  
}
```

- 4 Para almacenar datos, ubicaremos el archivo de "strings.xml" y "colors.xml" y agregaremos los siguientes arreglos de datos.



## strings.xml

```
<string name="welcome_message">Bienvenido a mi app</string>
<string name="search">Buscar</string>

<string-array name="dish_names">
    <item>Hamburguesa Clásica</item>
    <item>Papas Fritas</item>
    <item>Taco de Pollo</item>
    <item>Refresco</item>
</string-array>

<string-array name="dish_descriptions">
    <item>Jugosa hamburguesa con queso, lechuga y tomate.</item>
    <item>Crujientes papas doradas, servidas calientes.</item>
    <item>Taco de pollo sazonado con cebolla y cilantro.</item>
    <item>Refresco de 350 ml.</item>
</string-array>

<string-array name="dish_prices">
    <item>$3.50</item>
    <item>$1.50</item>
    <item>$2.00</item>
    <item>$0.80</item>
</string-array>

<!-- Nombres de las imágenes (sin extensión) -->
<string-array name="dish_images">
    <item>hamburguesa</item>
    <item>papas</item>
    <item>taco</item>
    <item>refresco</item>
</string-array>

<string-array name="dish_categories">
    <item>Platos fuertes</item>
    <item>Entradas</item>
    <item>Platos fuertes</item>
    <item>Bebidas</item>
</string-array>

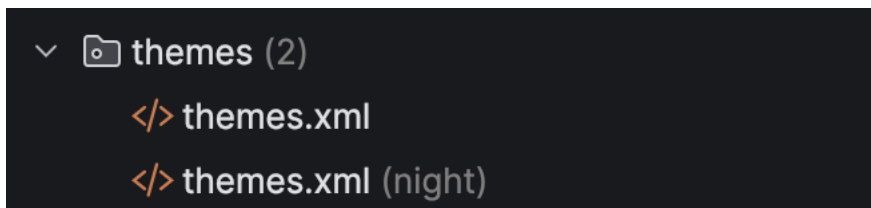
<string-array name="tab_categories">
    <item>Todos</item>
    <item>Entradas</item>
    <item>Platos fuertes</item>
    <item>Bebidas</item>
</string-array>

<plurals name="results_count">
    <item quantity="one">%d platillo encontrado</item>
    <item quantity="other">%d platillos encontrados</item>
</plurals>
```

colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFF</color>
    <color name="primaryColor">#6200EE</color>
    <color name="primaryDarkColor">#3700B3</color>
    <color name="accentColor">#03DAC5</color>
    <color name="cardBackground">#FFFFFF</color>
    <color name="textPrimary">#1B1B1B</color>
    <color name="textSecondary">#5F5F5F</color>
    <color name="primaryColorNight">#BB86FC</color>
    <color name="primaryDarkColorNight">#3700B3</color>
    <color name="accentColorNight">#03DAC5</color>
    <color name="cardBackgroundNight">#1E1E1E</color>
    <color name="textPrimaryNight">#F2F2F2</color>
    <color name="textSecondaryNight">#B0B0B0</color>
</resources>
```

- 5 Configuraremos los temas para el modo oscuro y claro del celular (Actualizar el Manifest)



Themes.xml

```
<resources>
    <style name="Theme.Catalogo"
parent="Theme.MaterialComponents.DayNight.NoActionBar">
        <item name="colorPrimary">@color/primaryColor</item>
        <item name="colorPrimaryVariant">@color/primaryDarkColor</item>
        <item name="colorSecondary">@color/accentColor</item>
        <item name="colorSurface">@color/cardBackground</item>
        <item name="colorOnSurface">@color/textPrimary</item>
        <item name="android:textColorSecondary">@color/textSecondary</item>
    </style>
</resources>
```

Themes.xml (night)

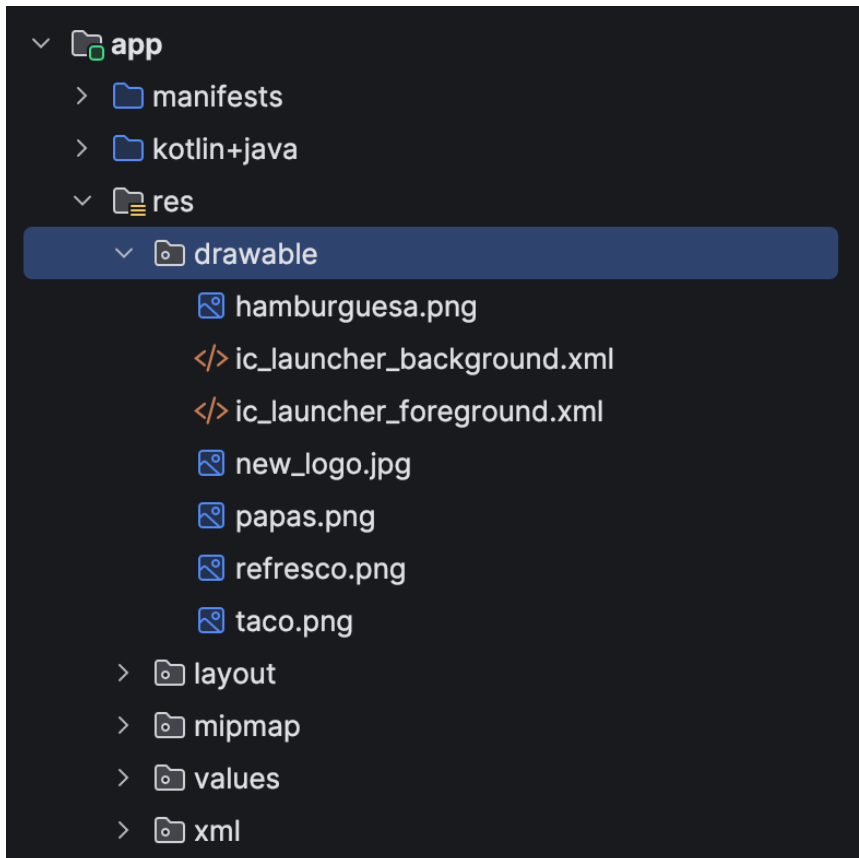
```
<resources>
    <style name="Theme.Catalogo"
parent="Theme.MaterialComponents.DayNight.NoActionBar">
        <item name="colorPrimary">@color/primaryColorNight</item>
        <item name="colorPrimaryVariant">@color/primaryDarkColorNight</item>
        <item name="colorSecondary">@color/accentColorNight</item>
    </style>
</resources>
```

```

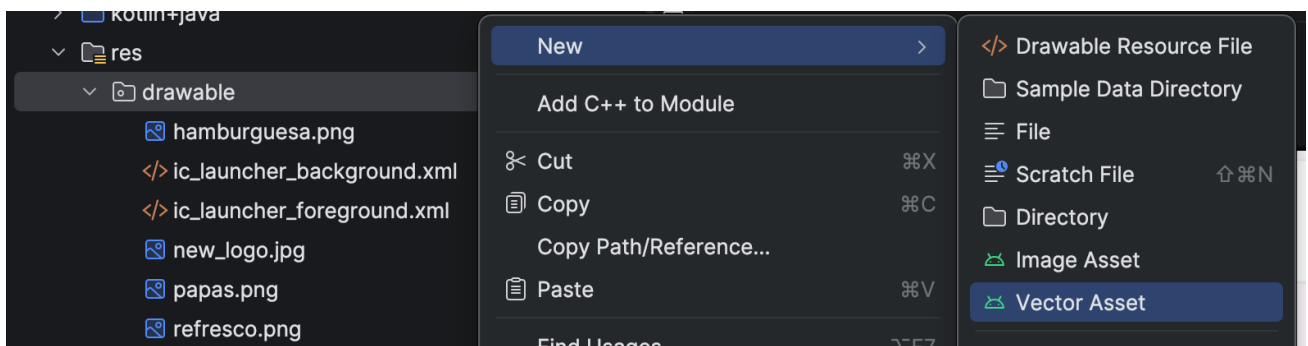
        <item name="colorSurface">@color/cardBackgroundNight</item>
        <item name="colorOnSurface">@color/textPrimaryNight</item>
        <item
name="android:textColorSecondary">@color/textSecondaryNight</item>
    </style>
</resources>

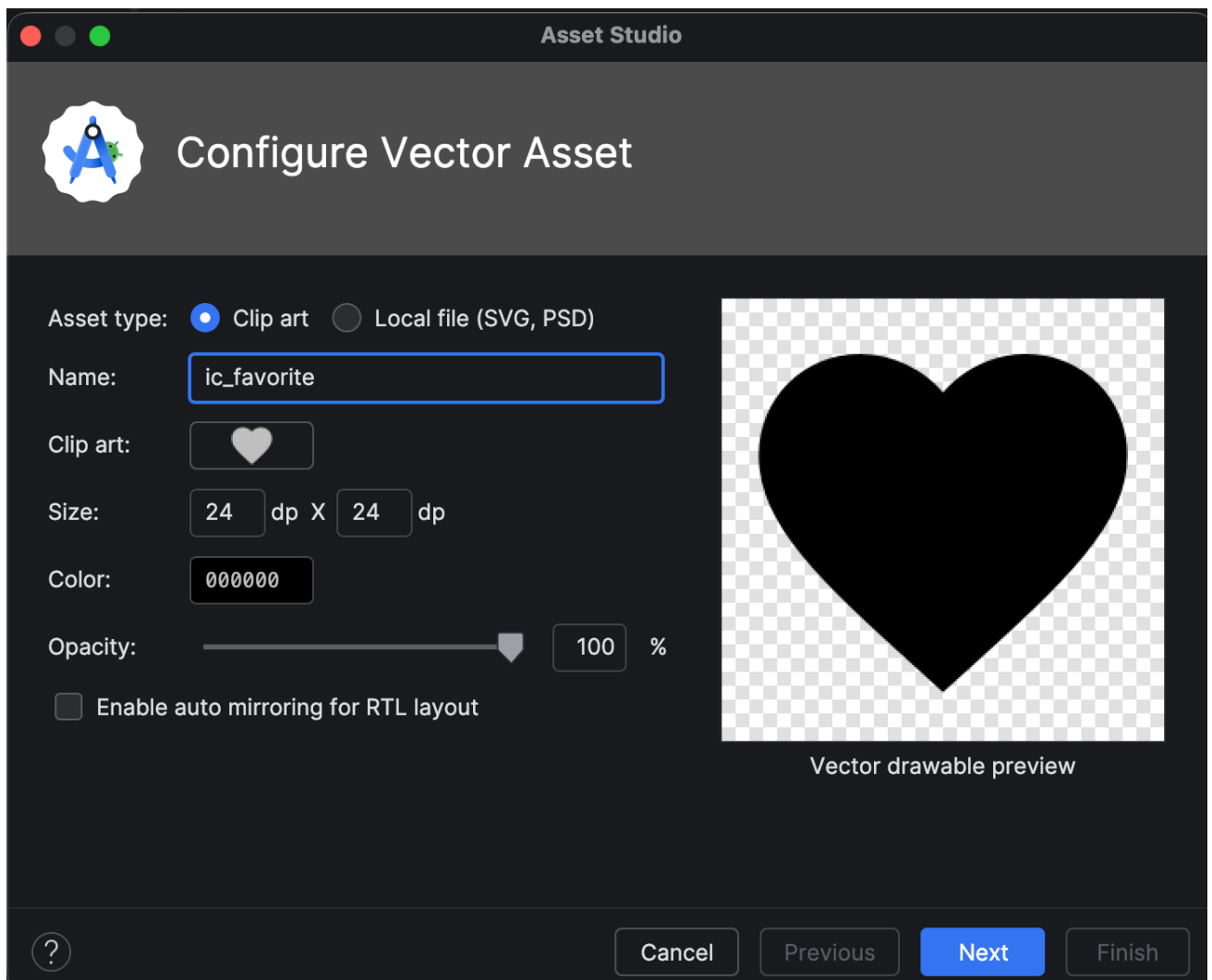
```

- 6 Procederemos a descargar las imágenes brindadas por el docente y colocarlas en la ubicación: app/src/main/res/drawable/



- 7 Genera dos vectores con New → Vector Asset (buscar "favorite" y "favorite\_border" en el catálogo de Material Icons): ic\_favorite.xml e ic\_favorite\_border.xml





- 8 En el “activity\_main” crearemos un recyclerView que mostrará el catálogo de comidas.

Activity\_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
```

```

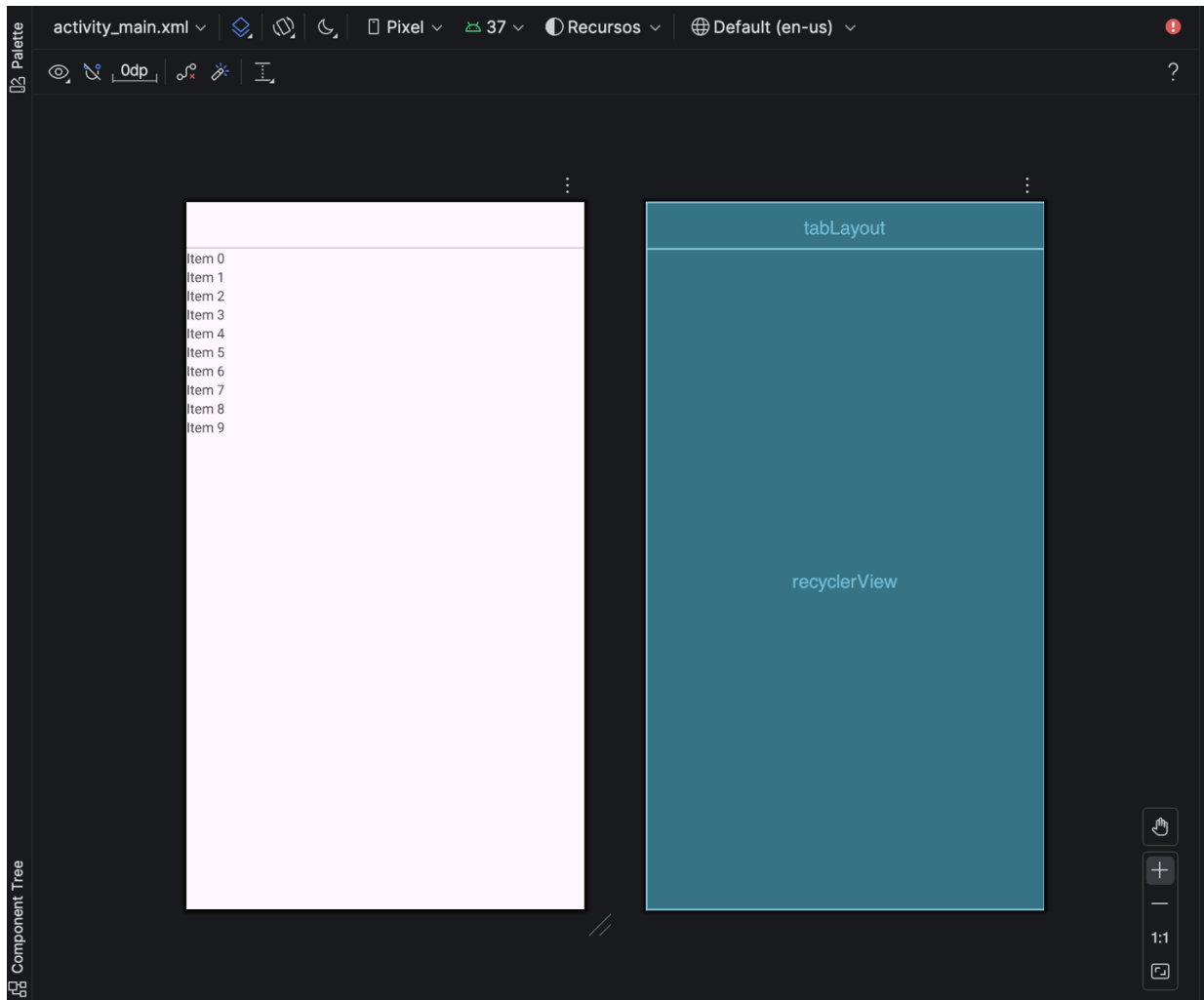
        android:layout_width="0dp"
        android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorPrimary"
        app:titleTextColor="@color/white"
        app:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"
        app:popupTheme="@style/ThemeOverlay.MaterialComponents.Light"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

<com.google.android.material.tabs.TabLayout
    android:id="@+id/tabLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:tabMode="scrollable"
    app:layout_constraintTop_toBottomOf="@id/toolbar"/>

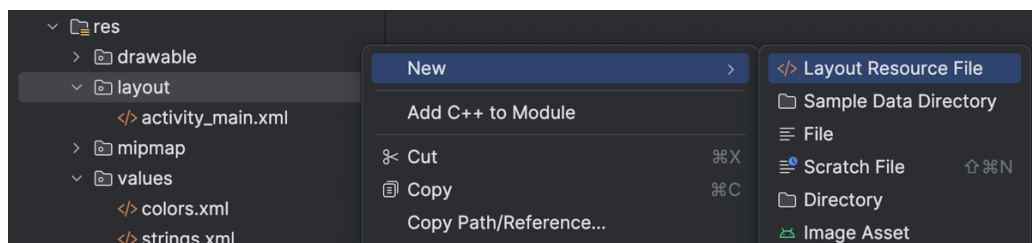
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    app:layout_constraintTop_toBottomOf="@id/tabLayout"
    app:layout_constraintBottom_toBottomOf="parent"/>

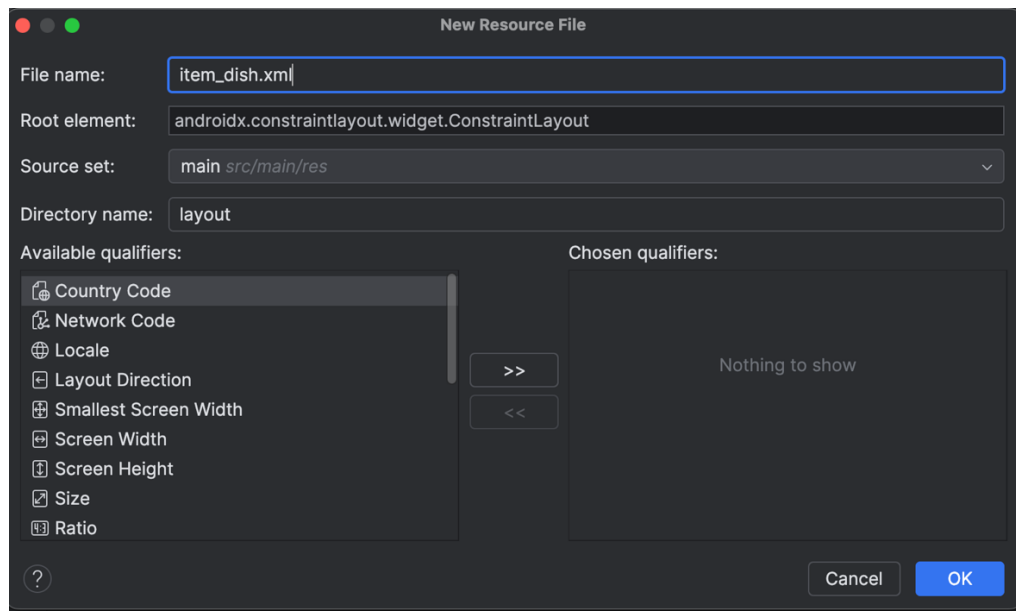
</androidx.constraintlayout.widget.ConstraintLayout>

```



- 9 Crearemos un nuevo archivo xml en la carpeta de “layout”, este elemento nuevo nos servirá para crear una plantilla para cada platillo, en la cual se mostrará en una tarjeta con imagen, nombre y precio.





10 En res/layout/item\_dish.xml crea un diseño con CardView, dos ImageView y dos TextView

item\_dish.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    android:clickable="true"
    app:cardBackgroundColor="?attr/colorSurface"
    android:foreground="?attr/selectableItemBackground">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:padding="8dp">

        <ImageView
            android:id="@+id/ivDish"
            android:layout_width="80dp"
            android:layout_height="80dp"
            android:scaleType="centerCrop"
            android:contentDescription="@string/app_name"/>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
```

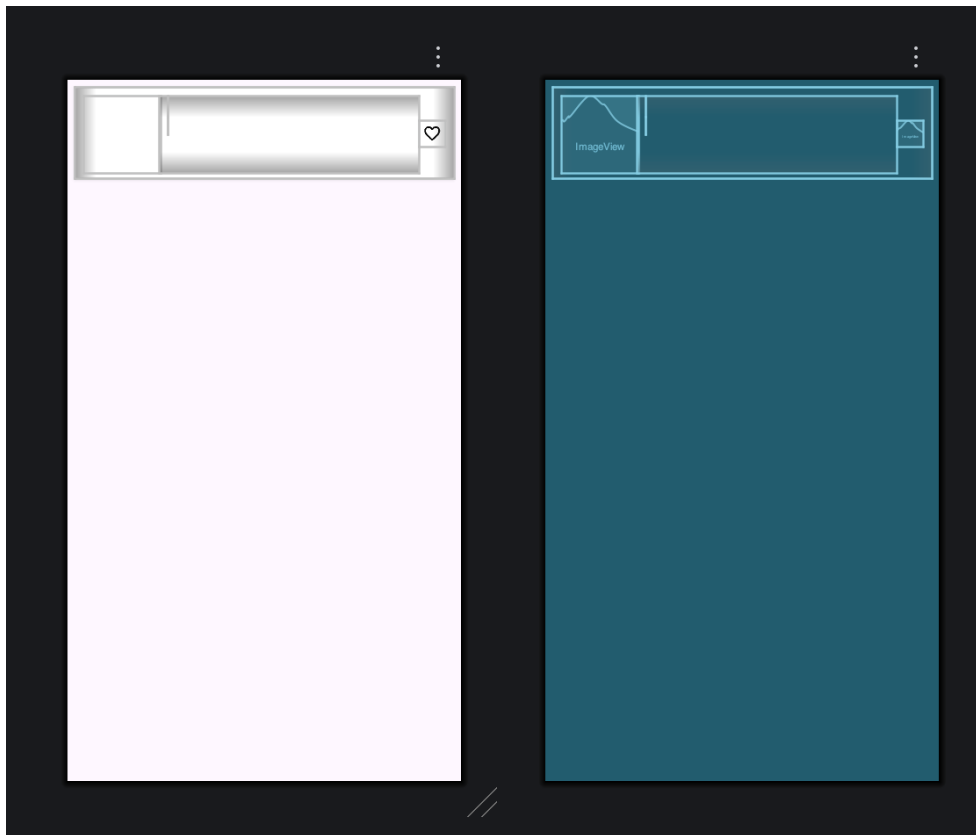
```
        android:orientation="vertical"
        android:paddingStart="8dp">

        <TextView
            android:id="@+id/tvName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="?attr/colorOnSurface"
            android:textSize="16sp"
            android:textStyle="bold"/>

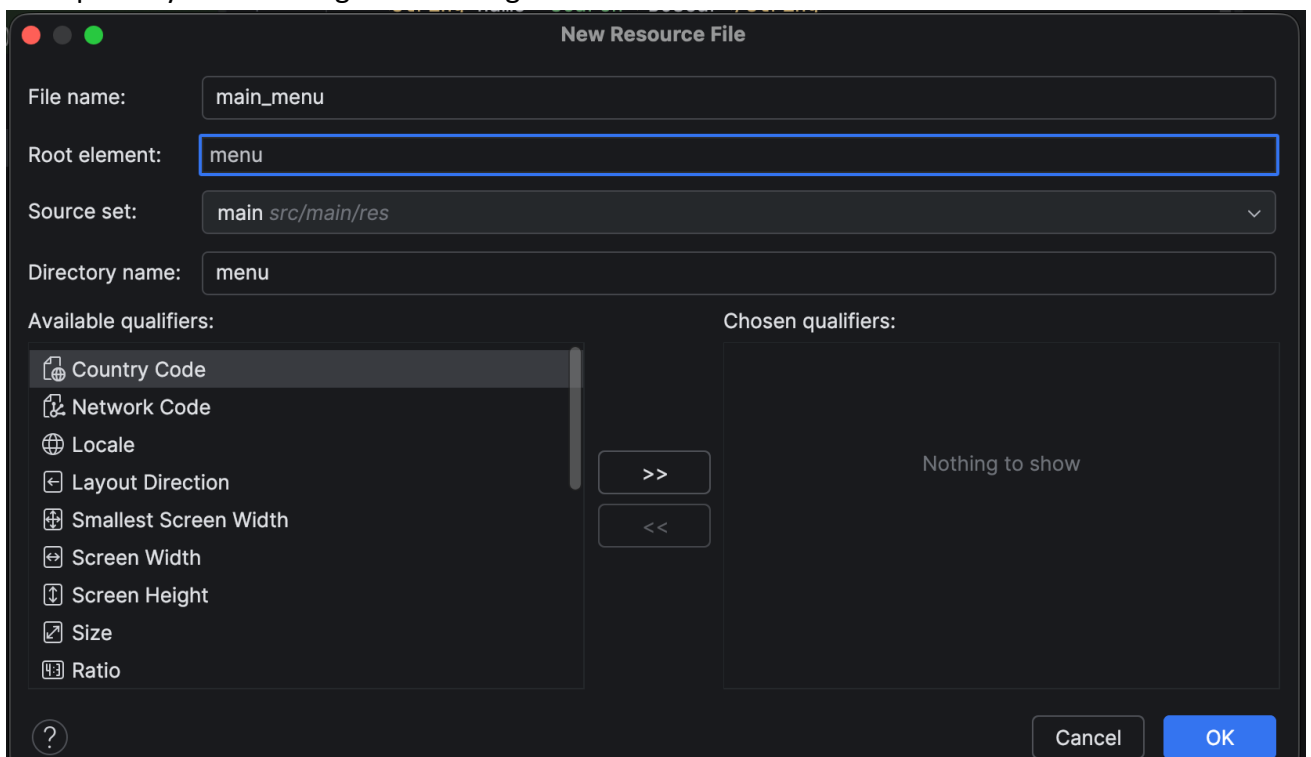
        <TextView
            android:id="@+id/tvPrice"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="?attr/android:textColorSecondary"
            android:textSize="14sp"/>
    </LinearLayout>

    <ImageView
        android:id="@+id/ivFavorite"
        android:layout_width="28dp"
        android:layout_height="28dp"
        android:padding="4dp"
        android:src="@drawable/ic_favorite_border"
        android:contentDescription="@string/app_name"/>
</LinearLayout>
</androidx.cardview.widget.CardView>
```





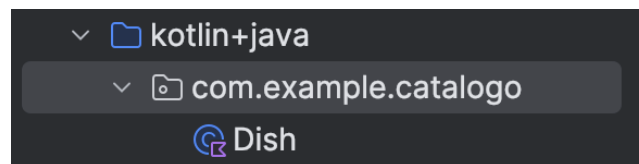
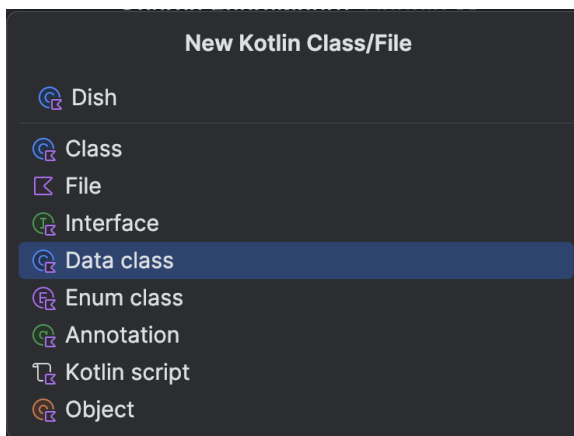
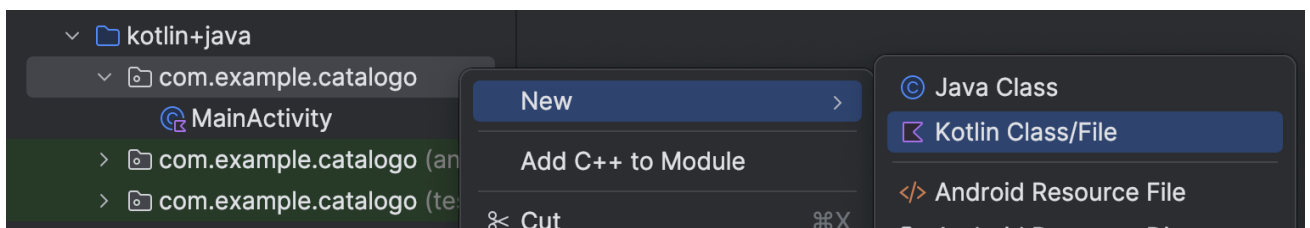
- 11 Crearemos un menú para poder filtrar la búsqueda de los ítems, deberá crear un recurso xml en la carpeta layout con la siguiente configuración.



Luego le agregaremos el siguiente código:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
    <item
        android:id="@+id/action_search"
        android:title="@string/search"
        app:showAsAction="always"
        app:actionViewClass="androidx.appcompat.widget.SearchView"/>
</menu>
```

- 12 Para manejar los datos de los platillos crearemos una “Data class” la cual representará un platillo con sus propiedades para facilitar el manejo de datos.

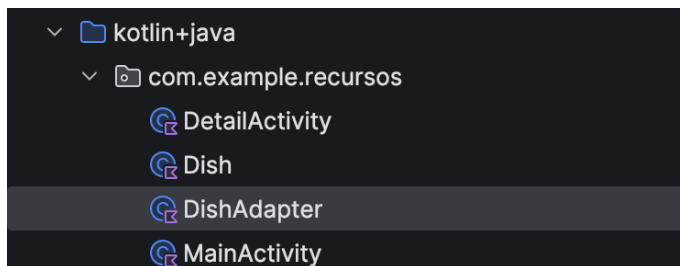
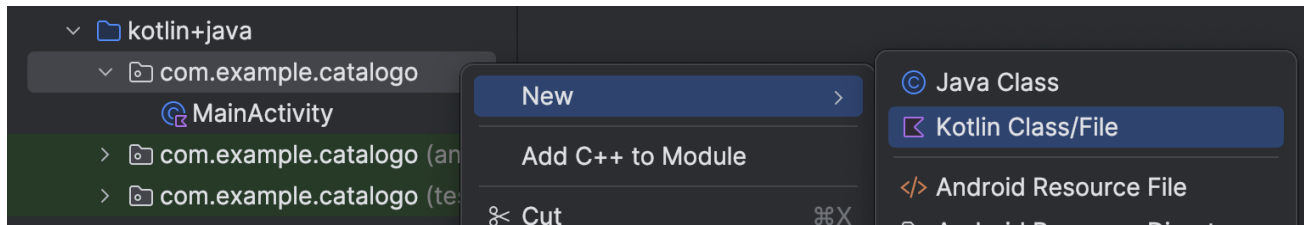


- 13 En la data class “Dish.kt” enlistaremos los campos que cada platillo tendrá, esto facilitará el manejo y la lectura de datos.

Dish.kt

```
data class Dish(
    val name: String,
    val description: String,
    val price: String,
    val imageName: String,
    val category: String,
    var isFavorite: Boolean = false
)
```

- 14 Para utilizar el RecyclerView en nuestro activity\_main deberemos de crear un adaptador para pasarle los datos y que renderize los datos que queremos presentar.



- 15 El adaptador enlaza la lista de Dish con el RecyclerView, inflando el diseño de cada elemento y manejando clics. En DishAdapter.kt crea un RecyclerView.Adapter.

#### DishAdapter.kt

```
class DishAdapter(  
    private var items: MutableList<Dish>,  
    private val onClick: (Dish) -> Unit,  
    private val onFavoriteToggle: (Dish) -> Unit  
) : RecyclerView.Adapter<DishAdapter.DishViewHolder>() {  
  
    inner class DishViewHolder(val binding: ItemDishBinding) :  
        RecyclerView.ViewHolder(binding.root)  
  
        override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
            DishViewHolder {  
                val binding =  
                    ItemDishBinding.inflate(LayoutInflater.from(parent.context), parent,  
                        false)  
                return DishViewHolder(binding)  
            }  
  
        override fun onBindViewHolder(holder: DishViewHolder, position: Int)  
        {  
            val dish = items[position]  
            val b = holder.binding  
            b.tvName.text = dish.name  
            b.tvPrice.text = dish.price  
  
            val ctx = holder.itemView.context
```

```

        val resId = ctx.resources.getIdentifier(dish.imageName,
"drawable", ctx.packageName)
        if (resId != 0) b.ivDish.setImageResource(resId)
        else b.ivDish.setImageResource(R.drawable.ic_launcher_foreground)

        b.ivFavorite.setImageResource(
            if (dish.isFavorite) R.drawable.ic_favorite else
R.drawable.ic_favorite_border
        )
        b.ivFavorite.setOnClickListener {
            dish.isFavorite = !dish.isFavorite
            notifyItemChanged(position)
            onFavoriteToggle(dish)
        }

        holder.itemView.setOnClickListener { onClick(dish) }
    }

    override fun getItemCount(): Int = items.size

    fun updateList(newList: List<Dish>) {
        items = newList.toMutableList()
        notifyDataSetChanged()
    }

    fun removeAt(position: Int): Dish {
        val removed = items.removeAt(position)
        notifyItemRemoved(position)
        return removed
    }

    fun insertAt(position: Int, dish: Dish) {
        items.add(position, dish)
        notifyItemInserted(position)
    }
}

```

### DishAdapter.kt — el "traductor" entre datos y pantalla

El RecyclerView no sabe nada de tus platillos; solo sabe dibujar filas reciclando vistas.

El **Adapter** es el intermediario que le dice qué mostrar en cada fila.

- **onCreateViewHolder**: se llama solo unas pocas veces (las necesarias para llenar la pantalla), infla el layout item\_dish.xml una vez por cada "hueco" visible y lo envuelve en un ViewHolder — así Android no vuelve a buscar los findViewById/binding cada vez que haces scroll, los guarda en memoria.

- **onBindViewHolder**: se llama **cada vez** que una fila reciclada necesita mostrar un platillo distinto (por ejemplo, al hacer scroll). Aquí es donde tomas el Dish de la posición actual y "pintas" sus datos en las vistas del ViewHolder (nombre, precio, imagen, ícono de favorito).
- **getItemCount**: le dice al RecyclerView cuántas filas hay en total, para que sepa cuándo dejar de pedir más.
- **updateList / removeAt / insertAt**: funciones tuyas (no del framework) para cambiar la lista interna del adapter cuando filtras, buscas o eliminas — cada una llama a un notifyItem... que le avisa al RecyclerView exactamente qué cambió, para que solo redibuje esa parte y no toda la lista.

16 Ya con el adaptador podremos realizar la lectura de los datos, procederemos a modificar el archivo "MainActivity.kt".

Esta Activity leerá los datos de strings.xml, creará la lista de Dish y cargará el adaptador en el RecyclerView.

Procedimiento:

- Usa `resources.getStringArray()` para obtener nombres, descripciones, precios e imágenes.
- Llena un `MutableList<Dish>`.
- Asigna el adaptador al RecyclerView con `LinearLayoutManager`

MainActivity.kt

```
import android.content.Intent
import android.os.Bundle
import android.view.Menu
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.SearchView
import androidx.recyclerview.widget.ItemTouchHelper
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.google.android.material.snackbar.Snackbar
import com.google.android.material.tabs.TabLayout

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var adapter: DishAdapter
    private var allDishes = listOf<Dish>()
    private var currentCategory = "Todos"
    private var currentQuery = ""

    private val prefs by lazy { getSharedPreferences("favorites",
MODE_PRIVATE) }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}
```

```

binding = ActivityMainBinding.inflate(layoutInflater)
setContentView(binding.root)

setSupportActionBar(binding.toolbar)

allDishes = buildDishList()

adapter = DishAdapter(
    allDishes.toMutableList(),
    onClick = { dish -> openDetail(dish) },
    onFavoriteToggle = { dish -> saveFavorite(dish) }
)

binding.recyclerView.layoutManager = LinearLayoutManager(this)
binding.recyclerView.adapter = adapter

resources.getStringArray(R.array.tab_categories).forEach {
binding.tabLayout.addTab(binding.tabLayout.newTab().setText(it))
}
binding.tabLayout.addOnTabSelectedListener(object :
TabLayout.OnTabSelectedListener {
    override fun onTabSelected(tab: TabLayout.Tab) {
        currentCategory = tab.text.toString()
        applyFilters()
    }
    override fun onTabUnselected(tab: TabLayout.Tab) {}
    override fun onTabReselected(tab: TabLayout.Tab) {}
}))

attachSwipeToDelete(binding.recyclerView)
}

private fun buildDishList(): List<Dish> {
    val names = resources.getStringArray(R.array.dish_names)
    val descriptions =
resources.getStringArray(R.array.dish_descriptions)
    val prices = resources.getStringArray(R.array.dish_prices)
    val images = resources.getStringArray(R.array.dish_images)
    val categories =
resources.getStringArray(R.array.dish_categories)
    val favorites = prefs.getStringSet("favorite_names", emptySet())
?: emptySet()

    val size = listOf(names.size, descriptions.size, prices.size,
images.size, categories.size).minOrNull() ?: 0
    return (0 until size).map { i ->
        Dish(names[i], descriptions[i], prices[i], images[i],
categories[i], favorites.contains(names[i]))
    }
}

private fun applyFilters() {

```

```

        val filtered = allDishes.filter { dish ->
            (currentCategory == "Todos" || dish.category ==
currentCategory) &&
                dish.name.contains(currentQuery, ignoreCase = true)
        }
        adapter.updateList(filtered)
        title = resources.getQuantityString(R.plurals.results_count,
filtered.size, filtered.size)
    }

    private fun saveFavorite(dish: Dish) {
        val current = prefs.getStringSet("favorite_names",
emptySet())?.toMutableSet() ?: mutableSetOf()
        if (dish.isFavorite) current.add(dish.name) else
current.remove(dish.name)
        prefs.edit().putStringSet("favorite_names", current).apply()
    }

    private fun attachSwipeToDelete(recyclerView: RecyclerView) {
        val callback = object : ItemTouchHelper.SimpleCallback(
            0, ItemTouchHelper.LEFT or ItemTouchHelper.RIGHT
        ) {
            override fun onMove(rv: RecyclerView, vh:
RecyclerView.ViewHolder, target: RecyclerView.ViewHolder) = false

            override fun onSwiped(viewHolder: RecyclerView.ViewHolder,
direction: Int) {
                val position = viewHolder.adapterPosition
                val removed = adapter.removeAt(position)
                Snackbar.make(recyclerView, "${removed.name} eliminado",
Snackbar.LENGTH_LONG)
                    .setAction("Deshacer") { adapter.insertAt(position,
removed) }
                    .show()
            }
        }
        ItemTouchHelper(callback).attachToRecyclerView(recyclerView)
    }

    private fun openDetail(dish: Dish) {
        val intent = Intent(this, DetailActivity::class.java).apply {
            putExtra("name", dish.name)
            putExtra("description", dish.description)
            putExtra("price", dish.price)
            putExtra("imageName", dish.imageName)
        }
        startActivity(intent)
    }

    override fun onCreateOptionsMenu(menu: Menu): Boolean {
        menuInflater.inflate(R.menu.main_menu, menu)
        val searchItem = menu.findItem(R.id.action_search)
        val searchView = searchItem.actionView as SearchView
    }

```

```

        searchView.setOnQueryTextListener(object :
SearchView.OnQueryTextListener {
            override fun onQueryTextSubmit(query: String?) = false
            override fun onQueryTextChange(newText: String?): Boolean {
                currentQuery = newText ?: ""
                applyFilters()
                return true
            }
        })
        return true
    }
}

```

### MainActivity.kt — la pantalla principal

Piensa en onCreate como una lista de tareas de arranque, en este orden:

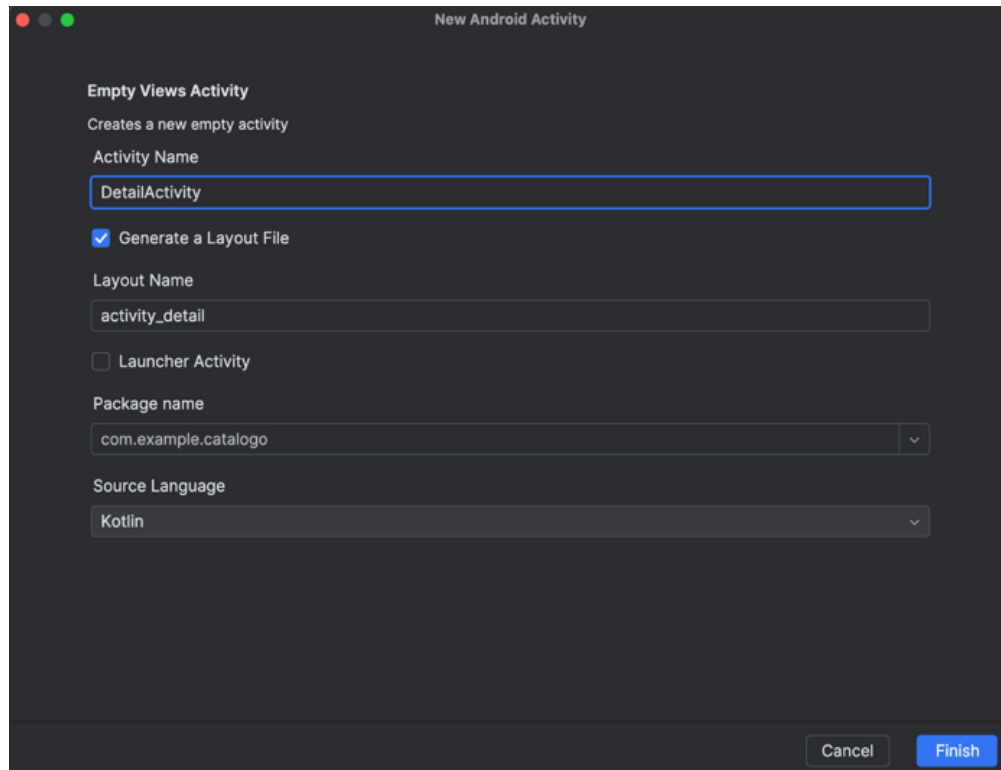
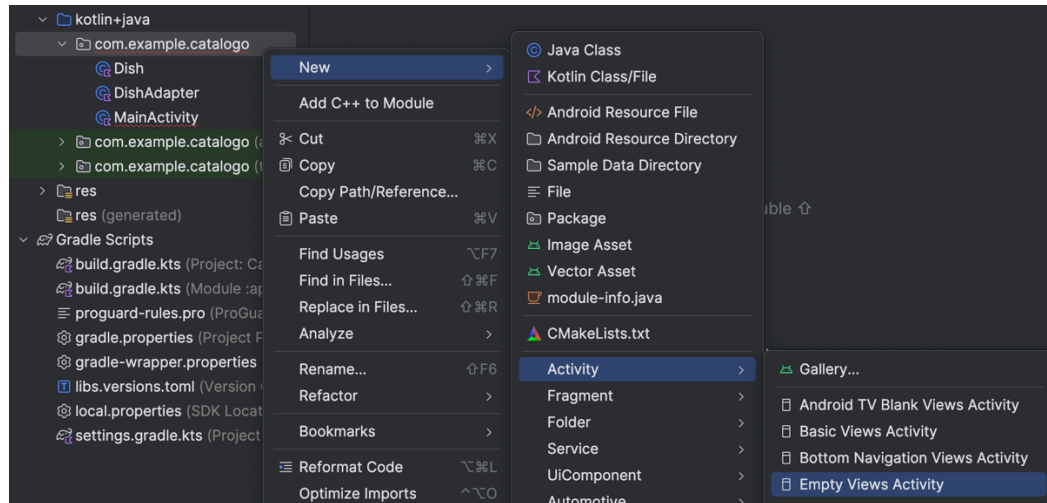
1. **Infla el layout** con ViewBinding (ActivityMainBinding) y lo muestra.
2. **setSupportActionBar(binding.toolbar)**: conecta tu Toolbar personalizado al ciclo de vida de menús de la Activity (sin esto, onCreateOptionsMenu nunca se dispara).
3. **buildDishList()**: lee los string-array de strings.xml y los combina en una lista de objetos Dish, cruzando también qué platillos están marcados como favoritos en SharedPreferences.
4. **Crea el DishAdapter** pasándole la lista y dos funciones lambda (onClick, onFavoriteToggle) — así el adapter no necesita saber *qué hacer* cuando tocan un platillo o un corazón, solo avisa "esto pasó" y la Activity decide la acción real (abrir detalle, guardar favorito).
5. **Configura las pestañas** (TabLayout) leyendo tab\_categories, y escucha cuando cambias de pestaña para volver a filtrar.
6. **attachSwipeToDelete**: activa el gesto de deslizar para eliminar con ItemTouchHelper.

Funciones de apoyo:

- **applyFilters()**: es el "cerebro" del filtrado — cruza la pestaña activa + el texto buscado, y le pasa la lista resultante al adapter. Se llama tanto al cambiar de pestaña como al escribir en el buscador.
- **saveFavorite()**: persiste el conjunto de nombres favoritos en SharedPreferences, para que sobrevivan si cierras la app.
- **onCreateOptionsMenu()**: infla el menú (la lupa) y conecta el SearchView para que cada letra escrita dispare applyFilters().



17 Crearemos la nueva actividad “DetailActivity”.



```

<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Catalogo">
    <activity
        android:name=".DetailActivity"
        android:exported="false" />
    <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>

```

Automaticamente la actividad será declarada en el “manifest” y se crearán los archivos DetailActivity.kt y activity\_detail.xml

## 18 Crearemos una interfaz para mostrar los datos en la actividad de detalles

activity\_detail.xml

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <ImageView
            android:id="@+id/ivDish"
            android:layout_width="match_parent"

```

```

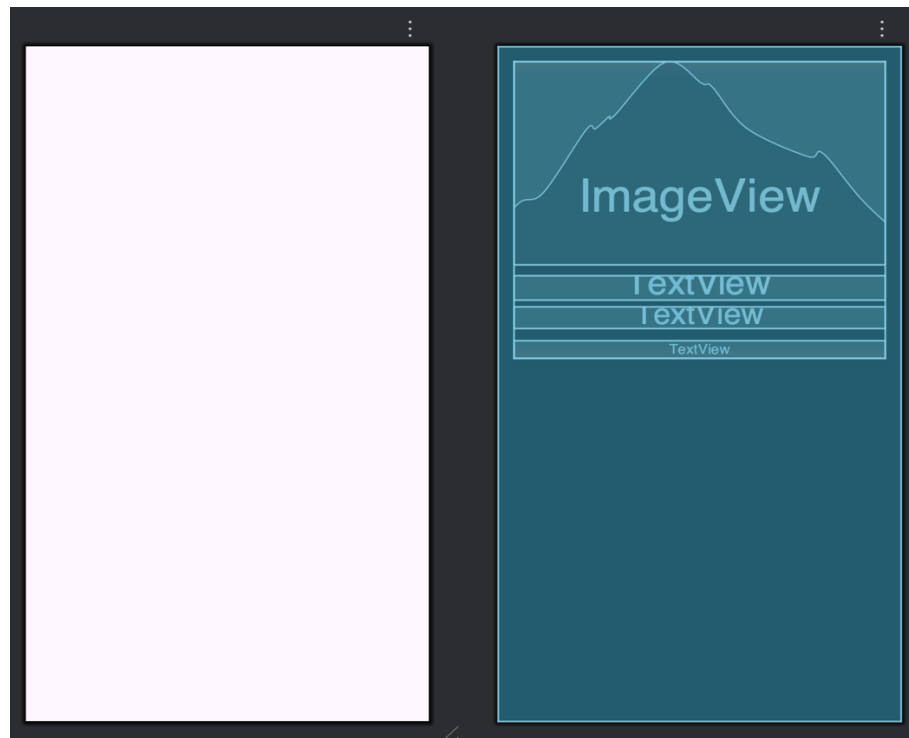
        android:layout_height="420dp"
        android:scaleType="centerCrop"
        android:contentDescription="@string/app_name"/>

<TextView
    android:id="@+id/tvName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="?attr/colorOnSurface"
    android:textSize="20sp"
    android:textStyle="bold"
    android:layout_marginTop="12dp"/>

<TextView
    android:id="@+id/tvPrice"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="?attr/android:textColorSecondary"
    android:textSize="18sp"
    android:layout_marginTop="6dp"/>

<TextView
    android:id="@+id/tvDescription"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="?attr/colorOnSurface"
    android:layout_marginTop="12dp"/>
</LinearLayout>
</ScrollView>

```



19 Luego modificaremos el controlador de la activity de detalles.

DetailActivity.kt

```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class DetailActivity : AppCompatActivity() {

    private lateinit var binding: ActivityDetailBinding

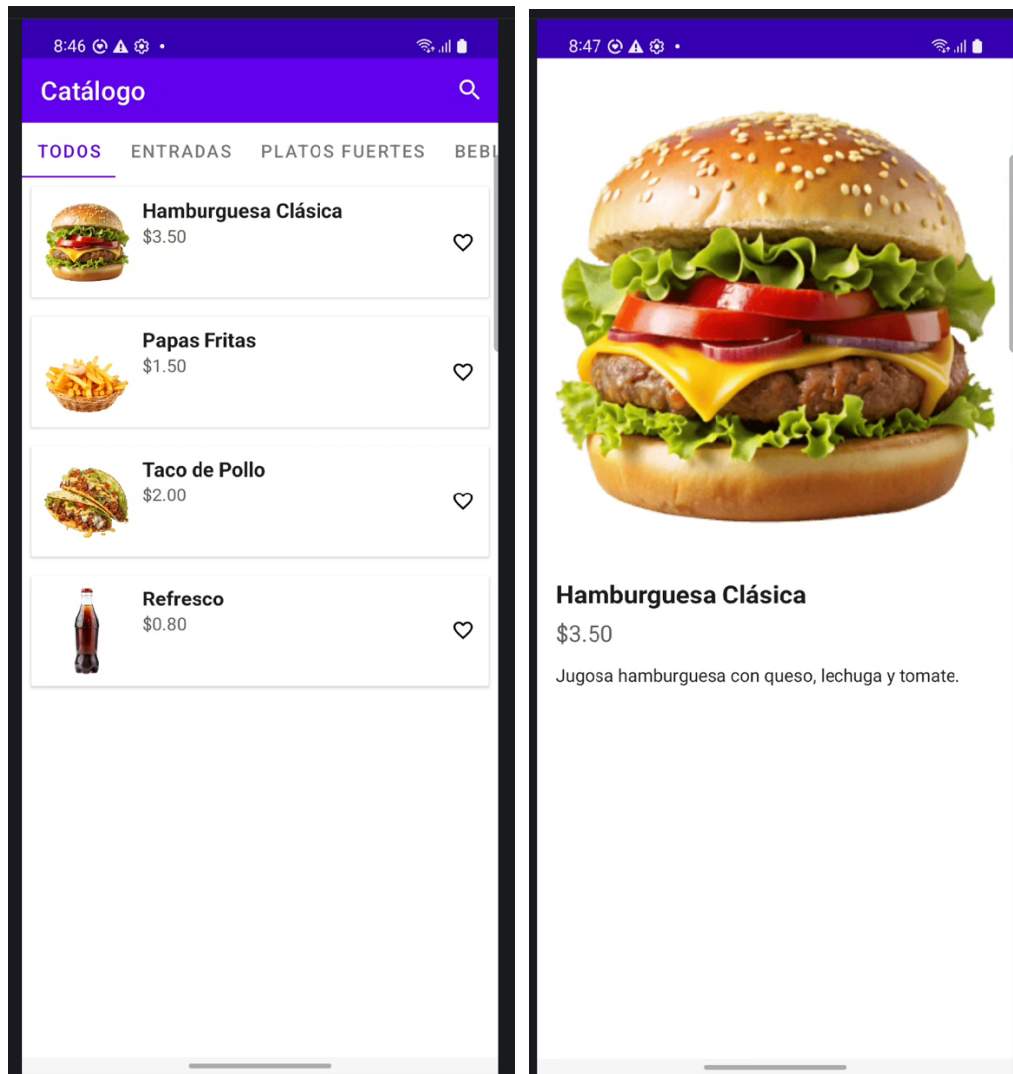
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityDetailBinding.inflate(layoutInflater)
        setContentView(binding.root)

        val name = intent.getStringExtra("name") ?: ""
        val description = intent.getStringExtra("description") ?: ""
        val price = intent.getStringExtra("price") ?: ""
        val imageName = intent.getStringExtra("imageName") ?: ""

        binding.tvName.text = name
        binding.tvDescription.text = description
        binding.tvPrice.text = price

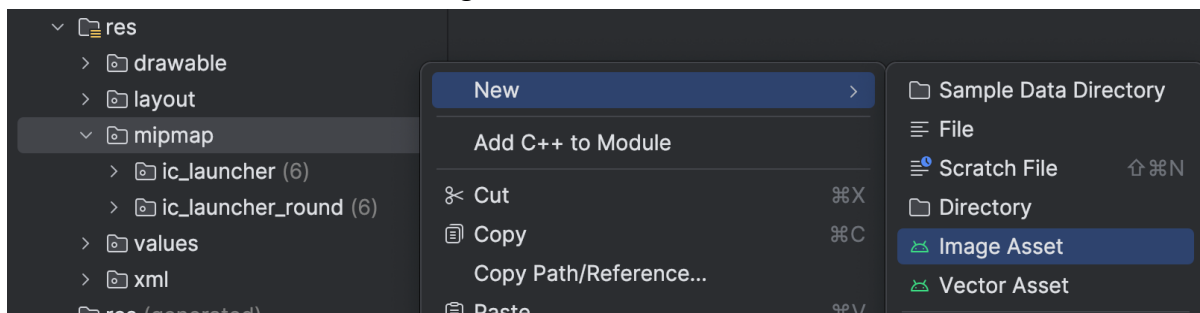
        val resId = resources.getIdentifier(imageName, "drawable",
packageName)
        if (resId != 0) binding.ivDish.setImageResource(resId)
        else
binding.ivDish.setImageResource(R.drawable.ic_launcher_foreground)
    }
}
```

20 Prueba la aplicación y el resultado final sería:

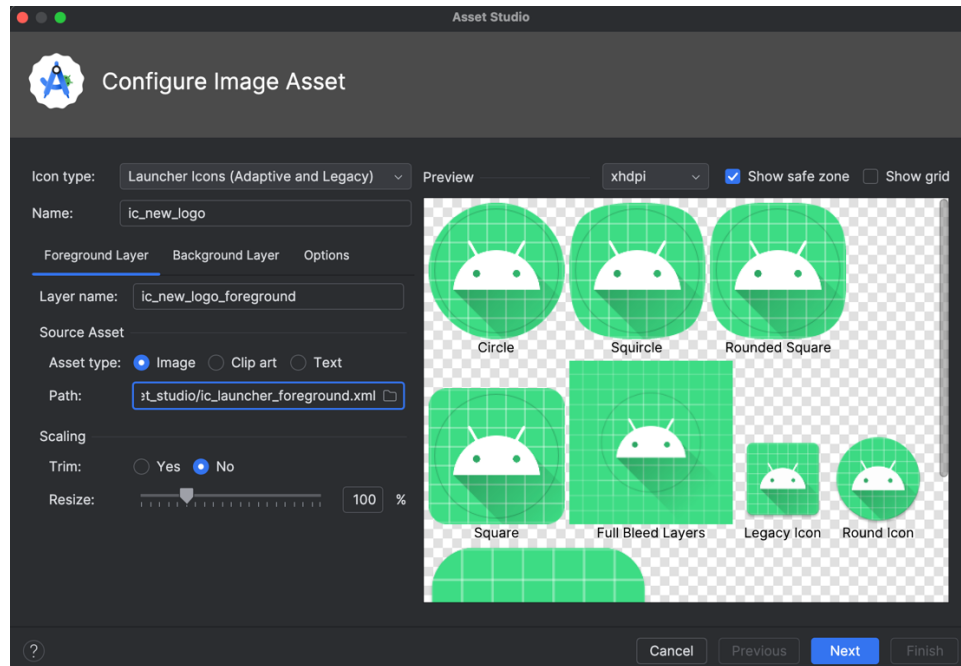


21 Como última parte cambiaremos el logo de la APP.

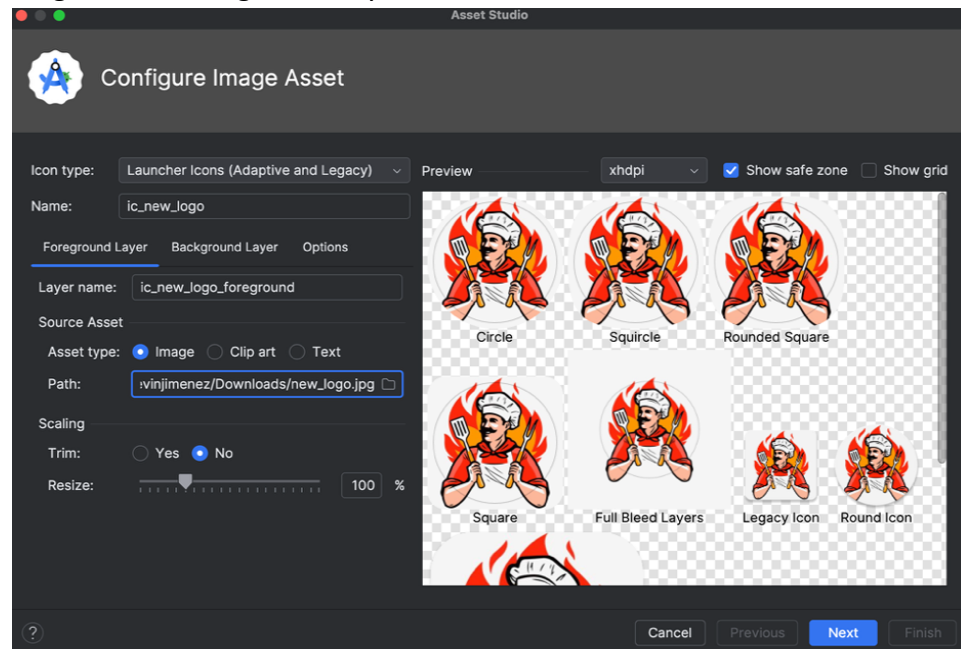
22 Click derecho en res → New → Image Asset.



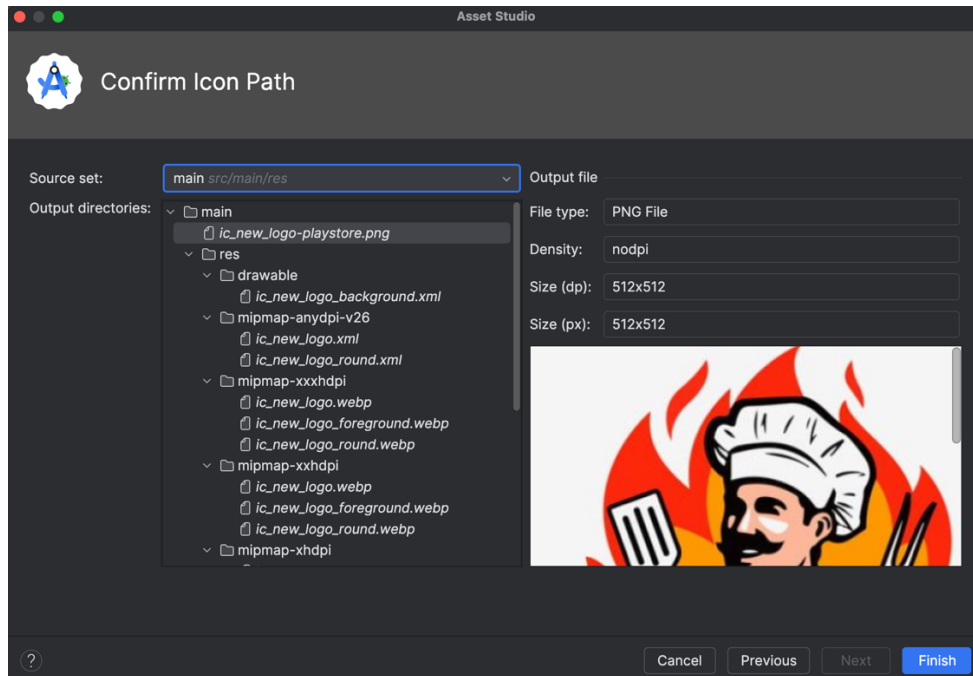
23 Selecciona Launcher Icons.



24 Carga tu imagen en el Foreground Layer.



25 Ajusta y finaliza para que se genere ic\_new\_logo en mipmap.



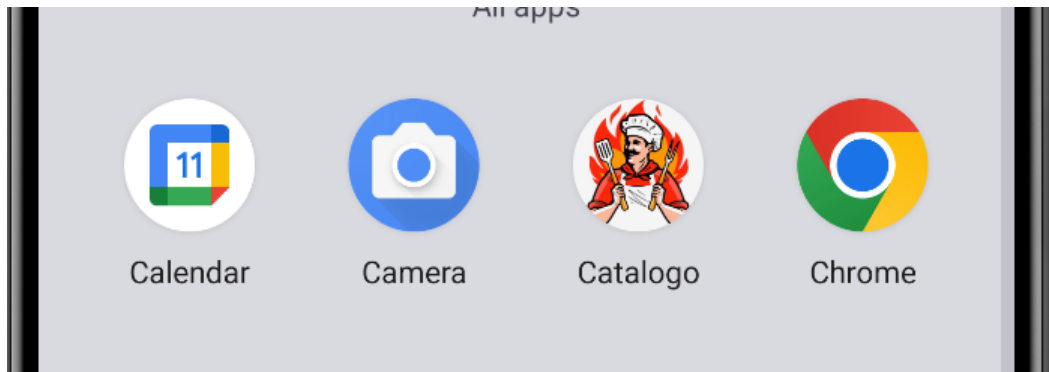
26 Cambiar el icono en el manifest de la aplicación (ic\_launcher -> ic\_new\_logo)

```

5      <application
6          android:allowBackup="true"
7          android:dataExtractionRules="@xml/data_extraction_rules"
8          android:fullBackupContent="@xml/backup_rules"
9          android:icon="@mipmap/ic_new_logo"
10         android:label="Catalogo"
11         android:roundIcon="@mipmap/ic_new_logo_round"
12         android:supportsRtl="true"
13         android:theme="@style/Theme.Catalogo">

```

27 El resultado será el siguiente:



28 Agrega 5 items de comida por cada categoría (Plato Fuerte, Entrada y Bebida)

## VII. BIBLOGRAFÍA

- Documentación Oficial Android - Descripción general sobre los recursos de las apps.  
<https://developer.android.com/guide/topics/resources/providing-resources?hl=es-419>